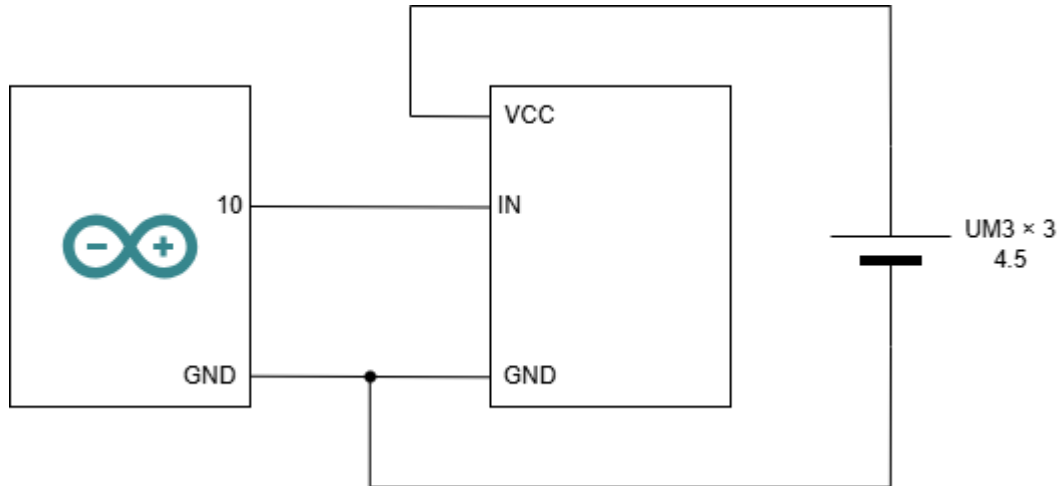


課題 4-1A: 2つのスイッチでモータの回転方向を制御する

1. 回路仕様

- **配線:** Arduino UNO ボードの 10番ピンにサーボモータの制御ピンを接続、サーボモータのVCCを外部電源である電池ボックスの+に接続、GNDをArduinoと電池ボックスのGNDに接続



2. 実装コード

```
int SERVO_PIN = 10; // サーボモータの制御ピンの番号をグローバル変数で定義

void setup()
{
    pinMode(SERVO_PIN, OUTPUT);
}

void servo_set(int theta, int period)
{
    // テキストの式:  $A = 500 + (1900 * \theta) / 180$  [us]
    long A = 500 + (1900 * (long)theta) / 180; // パルス幅を計算
    long low_time = 20000 - A; // LOW時間を計算 (20ms周期)

    int cycles = period * 50; // 20ms周期のため1秒間に50回パルスを送信

    for (int i = 0; i < cycles; i++)
    {
        digitalWrite(SERVO_PIN, HIGH);
        delayMicroseconds(A); // HIGH時間を待機
        digitalWrite(SERVO_PIN, LOW);

        // delayMicrosecondsは最大約16,383usのため2回に分けて待機
        delayMicroseconds(low_time - 10000); // LOW時間を待機
        delayMicroseconds(10000); // LOW時間を待機
    }
}

void loop()
{
    servo_set(0, 2); // 0度で2秒間保持
    servo_set(90, 1); // 90度で1秒間保持
    servo_set(180, 2); // 180度で2秒間保持
    servo_set(90, 1); // 90度で1秒間保持
}
```

3. 解説

① パルス幅制御の原理と計算式の導出

- **PWMによる角度指定:** サーボモータは周期 $T = 20 \text{ ms}$ (周波数 50 Hz) のパルス信号を受け取り、パルスのHIGH時間 (パルス幅 A) に応じて回転角度が決まる。
- **仕様値:** 本実験のサーボモータはパルス幅 $500 \mu\text{s}$ のとき 0° 、パルス幅 $2400 \mu\text{s}$ のとき 180° となる。
- **テキスト掲載の式 (テキスト p.16) :**

$$\theta = \theta_{min} + (\theta_{max} - \theta_{min}) \times \frac{A - 500}{2400 - 500}$$

$\theta_{min} = 0$ 、 $\theta_{max} = 180$ を代入すると、以下の関係になる。

$$\theta = 180 \times \frac{A - 500}{1900}$$

- **パルス幅 A の逆算:** 関数 `servo_set(theta, period)` では指定された目標角度 θ からマイコンが出力すべきパルス幅 A [μs] を求める必要があるため、式を A について変形する。

$$\begin{aligned}\frac{\theta}{180} &= \frac{A - 500}{1900} \\ A - 500 &= \frac{1900 \times \theta}{180} \\ A &= 500 + \frac{1900 \times \theta}{180}\end{aligned}$$

② ループ回数 (cycles) の計算

- 周期 $T = 20$ ms のパルスを1秒間に送信する回数は $1000 \text{ ms} / 20 \text{ ms} = 50$ 回である。
- したがって、指定時間 `period` [秒] だけ姿勢を保持するためには、パルス出力を `period * 50` 回繰り返す。

4. コードの詳細実装に対する技術的考察 (TA向け)

提示したコードには、マイコンのハードウェア制約を踏まえた配慮が2点組み込まれている。

- **long 型キャストの必要性 (必須):** Arduino UNO (ATmega328P) の `int` 型は16ビット符号付き整数 ($-32,768 \sim 32,767$) である。もし `int A = 500 + (1900 * theta) / 180;` と記述した場合、例えば $\theta = 90$ で計算途中の $1900 \times 90 = 171,000$ が16ビットの上限を超えてオーバーフローを起こし、意図しない数値になってサーボが暴走する。これを防ぐために `(long)theta` や `1900L` を用いて32ビット演算を行わせる処理は必須である。
- **delayMicroseconds() の分割処理 (簡略化可能):** Arduinoの `delayMicroseconds()` は仕様上、引数として渡せる最大値が $16,383 \mu s$ に制限されている。周期 $20,000 \mu s$ のうちLOW側の時間は約 $17,600 \sim 19,500 \mu s$ となり上限を超えるため、コード例では2回に分割して待機している。

5. シンプルに書く場合の別解 (テキスト list 4-1A 準拠)

LOW時間を分割する代わりに、テキストp.17の `list 4-1A` と同様に「ミリ秒待機の `delay(17)`」と「マイクロ秒待機の `delayMicroseconds(3000 - A)`」を組み合わせることで、特殊な分割記述を行わずに分かりやすく実装できる。

```
void servo_set(int theta, int period)
{
    // オーバーフロー防止のため 32bit (long) で計算
    long A = 500 + (1900L * theta) / 180;

    int cycles = period * 50; // 1秒あたり50サイクル

    for (int i = 0; i < cycles; i++)
    {
        digitalWrite(SERVO_PIN, HIGH);
        delayMicroseconds(A);           // HIGH出力

        digitalWrite(SERVO_PIN, LOW);
        delayMicroseconds(3000 - A);     // 可変部 (3ms) の残り待機
        delay(17);                       // 固定部 (17ms) 待機
    }
}
```

周期 20 ms を「可変部 3 ms = 3,000 μ s」と「固定部 17 ms」に分けることで、`delayMicroseconds` の上限 (16,383 μ s) を回避できる。学生がこの構成で解答を作成した場合も、テキストの構成に忠実な満点の解答として扱う。